제공 수치 방법의 사용 안내

함수 호출과 수렴 결과 읽기

차 례

1 제공 방법을 호출하는 순서 **1**
1.1 한 별과 같은 질량의 두 별 1
1.2 전체 비교를 실행하고 재개한다 1
1.3 초기조건과 해 전체를 확인한다 2
1.4 실패를 성공 기록과 섞지 않는다 2
1.5 Fortran 에서 가져다 쓴다 2

1 제공 방법을 호출하는 순서

이 문서는 함수 사용 시 옆에 두는 짧은 안내다. EOS 의 열역학적 표현은 본문 제 3·7·9 장, 양방향 경계값 문제는 제 4·5 장, 모드 식별과 수렴은 제 6·10 장에서 전개한다. 여기의 명령만 실행했다고 그 물리적 설명을 대신한 것은 아니다.

1.1 한 별과 같은 질량의 두 별

```
from tools.stellar_methods import Model, solve_star, solve_pair

star = solve_star("../one_star", alpha=0.0, level="fine")
pair = solve_pair("../one_pair", alpha=0.1, level="fine")
print(pair["delta_f_hz"], pair["delta_p1_hz"])
print(pair["physical_claim_status"])
```

기본 `Model` 은 동봉한 native Hermite 표, 목표 질량 $2.644606257493485 \times 10^{30}$ kg, 양의 표면압력 10^{20} Pa 와 중심 시작점 1 m 를 사용한다. 두 모드는 $\ell = 2$ 의 f, p_1 이다. `solve_pair` 는 두 별을 같은 질량에 맞추고 실제 고유함수를 비교하지만 전체 수렴 실험을 수행하지 않는다. 상태는 `pending_additional_checks` 다.

`level` 은 `coarse`, `medium`, `fine`, `audit` 중 하나다. 상대 허용오차와 native 구간 최대 단계는 각각 $(10^{-7}, 0.2)$, $(10^{-9}, 0.1)$, $(10^{-11}, 0.05)$, $(10^{-13}, 0.025)$ 다. 단계 좌표는 구간별 무차원 좌표이며 반경 m 단위의 격자폭이 아니다. `method="rk4"` 는 별도의 단계법 점검이며 같은 상대오차 제어를 수행한다는 뜻이 아니다.

```
custom = Model(target_mass_kg=2.6e30, p_cut_pa=1e20, r0_m=1.0)
pair = solve_pair("../another_mass", model=custom)
```

이렇게 질량을 바꾼 결과에 기본 실험의 수렴 판정을 자동 부여하지 않는다. 새로운 문제의 branch 와 검색 구간, 모드 대응 및 경계 민감도를 다시 정해야 한다. 이 인터페이스가 임의의 EOS·질량에서 근의 존재나 스펙트럼 완전성을 보장하지는 않는다.

1.2 전체 비교를 실행하고 재개한다

```
from tools.stellar_methods import run_study
result = run_study("../my_study", max_new_runs=2)
print(result["status"], result["pending_runs"])
```

같은 호출을 반복하면 검증된 완료 사례는 다시 적분하지 않고 필요한 다음 사례로 간다. 한 번에 끝내려면 `max_new_runs` 를 생략한다. 이 제한은 한 호출의 작업 수만 바꾸며 물리·수치 허용오차를 낮추지 않는다.

전체 preset 은 21 개 단일별 계산이다. `main` 은 네 정밀도의 두 별, `method` 는 별도의 RK4 pair, `boundary` 는 중심·표면·연결점의 세 pair, `mass` 는 두 모델 각각의 질량 기여 추정에 쓰는 네 이웃, `null` 은 무변형 계산이다. 한 그룹만 실행해도 전체 필수 목록은 사라지지 않는다.

```
python tools/stellar_methods.py --output ../my_study --group main
python tools/stellar_methods.py --output ../my_study
```

`study.json` 은 현재 판정과 남은 사례, `convergence.csv` 는 두 절대 주파수와 차이, 각 사례 디렉터리는 입력·수치 출력·실행 상태를 담는다. 모두 끝나기 전에는 `pending` 이다. 최종 `numerically_resolved_in_fixed_model` 은 정의한 비교에서 수치적 분해가 확인되었다는 뜻이며 엄밀한 오차 상한이나 실제 EOS 의 물리적 불확실성 범위를 뜻하지 않는다.

1.3 초기조건과 해 전체를 확인한다

```
from tools.stellar_methods import Model, solve_pair
m = Model(r0_m=100.0, center_order=2)
pair = solve_pair("../series_start", model=m, level="audit")
```

`center_order=2` 는 제 3·5 장에서 유도한 다음 중심항을 제공 함수로 계산한다. 기본값 0 은 1 m 의 leading 시작이다. 큰 시작점이 다른 EOS 구간을 건너면 거절한다. 임의의 물리 우변에 보편적으로 적용되는 보정이 아니며, 새 설정의 결과에도 전체 수렴 판정이 자동 부여되지 않는다.

```
from tools.variational_check import audit_run
check = audit_run("../my_study/L3_a0.0", "../one_integral.json")
```

전체 `run_study` 는 네 paired 수준의 적분 검사를 자동 수행하고 `variational_checks.json` 에 저장한다. 주파수를 재대입하지 않은 scalar 항등식과 표면항은 본문 제 5·6 장에 있다. 적분 불일치가 수치 여유를 넘으면 `integral_consistency_unresolved` 로 남긴다. 원래 주파수를 적분값으로 덮어쓰지는 않는다.

1.4 실패를 성공 기록과 섞지 않는다

NaN·무한대, 잘못된 설정, 일치하지 않는 소스·입력·출력은 거절한다. 실패한 계산 폴더는 성공 결과로 사용하지 않는다. 같은 입력을 다시 시도하려면 명시적으로 다음을 사용한다.

```
python tools/stellar_methods.py --output ../my_study --retry-failed
```

원래 시도는 `failed_attempts/` 아래에 보존된다. 코드를 수정했거나 물리 입력을 바꿨으면 새 출력 폴더를 선택한다. 동일 출력 폴더의 동시 실행은 잠금으로 거절하며 중단되면 운영체제가 잠금을 해제한다. 이 재개 기능은 POSIX 환경을 사용한다.

1.5 Fortran 에서 가져다 쓴다

```
use ns_numerical_methods, only: adaptive_integrate, bisect
use ns_stellar_methods, only: integrate_cells, match_cells
```

첫 모듈은 수치 적분과 근 찾기, 둘째는 EOS 구간에서 TOV·Cowling 물리 함수를 연결하는 adapter 다. 실제 인수의 자료형은 해당 모듈과 `cell_solve.f90` 예제를 따른다. `methods` 에는 원래 EOS 표를 다루는 완성 함수도 있다. TOV·Cowling 식과 물리 경계조건은 `calculation/src/physics/` 에 별도로 보인다.

직접 구현한 물리 함수는 `make_calculation.py--physicslearner/src` 로 명시적으로 연결한다. 미완성 함수에 대한 자동 해설 대체는 없다. 제공 예제의 수치적 성공과 학생의 직접 구현 성취도 다른 판정이다.